LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 6: GENERATE DATA OTOMATIS & AUTHENTICATION



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum tugas Modul 6 ini adalah sebagai berikut:

1. Memahami authentication manual pada Laravel tanpa memakai Laravel Breeze.

2. Membuat fitur register, login, dan logout memakai RegisterController dan LoginController, lengkap dengan validasi input, Hash::make(), Auth::attempt(), dan Auth::logout().

3. Melindungi halaman daftar laporan dan halaman pengelolaan lain memakai middleware auth, sedangkan form pelaporan warga tetap dapat diakses tanpa login.

4. Menampilkan menu Login dan Register ketika pengguna belum login, serta menu Daftar Laporan, Profil Petugas, dan Logout ketika pengguna sudah login memakai directive @auth dan @endauth.

5. Membuat halaman profil petugas yang menampilkan nama, email, dan waktu pembuatan akun memakai Auth::user().

6. Membuat PetugasSeeder berbasis Faker dengan locale id_ID untuk menghasilkan lima akun petugas dummy dan satu akun admin BPBD dengan password yang di-hash.

7. Menguji alur authentication pada aplikasi LaporBanjir dan menganalisis hasilnya.

2. Alat dan Bahan

Praktikum ini dikerjakan menggunakan satu buah laptop dengan prosesor Intel Core i7-13650HX dan sistem operasi Windows 11 Pro. Perangkat lunak yang digunakan meliputi Laragon sebagai lingkungan pengembangan web lokal, PHP versi 8.3.30, Composer versi 2.10.3, Laravel Framework versi 13.37.0, SQLite versi 3.40.0 sebagai database, Visual Studio Code sebagai editor, Git sebagai version control, dan Google Chrome sebagai browser. Tampilan aplikasi memakai Bootstrap versi 5.3.0 dan Font Awesome versi 6.4.0 melalui CDN, sehingga Node.js dan NPM tidak diperlukan. Project yang dipakai adalah project Laravel baru di folder praktikum6-ta.

3. Langkah-Langkah Praktikum

Tugas praktikum yang saya kerjakan adalah nomor 1 pada bagian Tugas Praktikum Modul 6. BPBD Kabupaten Bandung meminta halaman pengelolaan laporan LaporBanjir hanya dapat dibuka oleh petugas yang sudah login, sedangkan warga tetap bisa mengirim laporan tanpa akun. Fitur yang dibutuhkan adalah authentication manual tanpa Breeze, proteksi halaman

daftar laporan, navigasi yang berubah sesuai status login, halaman profil petugas, dan seeder akun petugas.

Folder praktikum6-ta yang saya pakai berisi project Laravel baru. Karena itu, fitur LaporBanjir dari Modul 4 saya siapkan kembali lebih dulu, mulai dari migration, model, controller, route, sampai view form dan daftar laporan. Setelah aplikasi dasar berjalan, authentication manual ditambahkan di atasnya.

3.1 Menyiapkan Project LaporBanjir

Project dijalankan dengan php artisan serve di alamat http://127.0.0.1:8000. Database yang dipakai adalah SQLite melalui file database/database.sqlite, dan migration bawaan Laravel dijalankan dengan php artisan migrate. Aplikasi memiliki tiga alur utama. Form pelaporan berada di alamat / dan dapat dibuka tanpa login. Data form dikirim ke POST /laporan. Daftar laporan berada di GET /laporan dan hanya dapat dibuka setelah login. Halaman profil petugas berada di /profil dan juga dilindungi middleware auth.

Struktur data laporan mengikuti Modul 4, yaitu nama_pelapor, lokasi, tinggi_genangan, dan tanggal_kejadian. Status genangan ditentukan dari tinggi air: di bawah 30 cm berstatus Waspada, 30 sampai 70 cm berstatus Siaga, dan di atas 70 cm berstatus Bahaya.

3.2 Membuat Migration dan Model Laporan

Tabel laporan dibuat melalui file migration. Perintah php artisan make:migration create_laporan_table menghasilkan file baru di folder database/migrations. Isi method up() saya ubah menjadi seperti berikut.

Untitled-1

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('laporan', function (Blueprint $table) {
            $table->id();
            $table->string('nama_pelapor');
            $table->string('lokasi');
            $table->integer('tinggi_genangan');
            $table->date('tanggal_kejadian');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('laporan');
    }
};
```

Method up() membuat tabel laporan dengan kolom id sebagai primary key, nama_pelapor dan lokasi bertipe string, tinggi_genangan bertipe integer, tanggal_kejadian bertipe date, serta created_at dan updated_at dari timestamps(). Method down() menghapus tabel yang sama ketika migration di-rollback.

Model dibuat dengan php artisan make:model Laporan. File app/Models/Laporan.php kemudian diisi dengan properti $table, $fillable, dan cast tanggal_kejadian.

Untitled-1

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Laporan extends Model
{
    protected $table = 'laporan';

    protected $fillable = [
        'nama_pelapor',
        'lokasi',
        'tinggi_genangan',
        'tanggal_kejadian',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'tanggal_kejadian' => 'date',
        ];
    }
}
```

Properti $table diisi 'laporan' karena nama tabel tidak mengikuti bentuk jamak bahasa Inggris dari nama model. Properti $fillable berisi empat kolom yang boleh diisi lewat

Laporan::create(). Cast date membuat tanggal_kejadian otomatis menjadi objek Carbon, sehingga di view bisa diformat dengan format('d-m-Y').

3.3 Membuat Controller dan Route Laporan

Controller dibuat dengan php artisan make:controller LaporanController. Controller ini menangani form, penyimpanan, dan daftar laporan.

Untitled-1

```php
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;

class LaporanController extends Controller
{
    public function create()
    {
        return view('laporan.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama_pelapor' => ['required', 'string', 'max:255'],
            'lokasi' => ['required', 'string', 'max:255'],
            'tinggi_genangan' => ['required', 'integer', 'min:0'],
            'tanggal_kejadian' => ['required', 'date'],
        ]);

        $laporan = Laporan::create($validated);

        return view('laporan.konfirmasi', compact('laporan'));
    }

    public function index()
    {
        $laporans = Laporan::latest()->get();

        return view('laporan.index', compact('laporans'));
    }
}
```

Method create() menampilkan view laporan.create. Method store() memvalidasi nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian, lalu menyimpan data dengan Laporan::create(). Setelah tersimpan, method store() mengembalikan view konfirmasi bersama data laporan yang

baru dibuat. Method index() mengambil seluruh laporan dengan Laporan::latest() dan mengirimnya ke view laporan.index.

Route menghubungkan alamat URL dengan method controller. Isi routes/web.php menjadi seperti berikut.

Untitled-1

```
<?php

use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\Auth\RegisterController;
use App\Http\Controllers\LaporanController;
use App\Http\Controllers\ProfilController;
use Illuminate\Support\Facades\Route;

Route::get('/', [LaporanController::class, 'create'])->name('laporan.create');
Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');

Route::get('/register', [RegisterController::class, 'create'])->name('register');
Route::post('/register', [RegisterController::class, 'store']);
Route::get('/login', [LoginController::class, 'create'])->name('login');
Route::post('/login', [LoginController::class, 'store']);
Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');

Route::middleware('auth')->group(function () {
    Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');
    Route::get('/profil', [ProfilController::class, 'index'])->name('profil');
});
```

Route GET / dan POST /laporan berada di luar middleware auth karena form pelaporan harus tetap bisa dipakai warga. Route GET /laporan dan GET /profil dimasukkan ke dalam grup middleware auth. Route login diberi nama login karena middleware auth mengarahkan pengguna yang belum login ke route dengan nama tersebut. Route logout memakai method POST supaya tidak bisa dipicu hanya dengan membuka tautan.

3.4 Membuat Layout dan View LaporBanjir

Layout resources/views/layouts/app.blade.php memakai Bootstrap dan memuat navbar yang berubah sesuai status login. Bagian navbar ditulis dengan directive @auth dan @endauth.

Untitled-1

```
<nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
    <div class="container">
        <a class="navbar-brand" href="{{ route('laporan.create') }}">
            <i class="fas fa-water me-1"></i>LaporBanjir
        </a>
        <button class="navbar-toggler" type="button"
                data-bs-toggle="collapse" data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ms-auto">
                @auth
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('laporan.index') }}">
                            <i class="fas fa-list me-1"></i>Daftar Laporan
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('profil') }}">
                            <i class="fas fa-user me-1"></i>Profil Petugas
                        </a>
                    </li>
                    <li class="nav-item">
                        <form method="POST" action="{{ route('logout') }}">
                            @csrf
                            <button type="submit" class="nav-link btn btn-link">
                                <i class="fas fa-sign-out-alt me-1"></i>Logout
                            </button>
                        </form>
                    </li>
                @else
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('login') }}">
                            <i class="fas fa-sign-in-alt me-1"></i>Login
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('register') }}">
                            <i class="fas fa-user-plus me-1"></i>Register
                        </a>
                    </li>
                @endauth
            </ul>
        </div>
    </div>
</nav>
```

Ketika pengguna sudah login, navbar menampilkan Daftar Laporan, Profil Petugas, dan tombol Logout. Ketika belum login, navbar menampilkan Login dan Register. Tombol Logout berada di dalam form POST dengan @csrf karena route logout memakai method POST.

Form pelaporan memakai method POST, action route('laporan.store'), dan directive @csrf. Setiap input memakai old() agar nilai sebelumnya tidak hilang ketika validasi gagal, serta @error untuk menampilkan pesan galat di bawah input.

Untitled-1

```
<form method="POST" action="{{ route('laporan.store') }}">
    @csrf

    <div class="mb-3">
        <label for="nama_pelapor" class="form-label">Nama Pelapor</label>
        <input id="nama_pelapor" type="text" name="nama_pelapor"
               @class(['form-control', 'is-invalid' => $errors->has('nama_pelapor')])
               value="{{ old('nama_pelapor') }}" required>
        @error('nama_pelapor')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="row">
        <div class="col-md-6 mb-3">
            <label for="lokasi" class="form-label">Lokasi Kejadian</label>
            <input id="lokasi" type="text" name="lokasi"
                   @class(['form-control', 'is-invalid' => $errors->has('lokasi')])
                   value="{{ old('lokasi') }}" required>
            @error('lokasi')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="col-md-6 mb-3">
            <label for="tinggi_genangan" class="form-label">
                Tinggi Genangan (cm)
            </label>
            <input id="tinggi_genangan" type="number" name="tinggi_genangan" min="0"
                   @class(['form-control', 'is-invalid' => $errors->has('tinggi_genangan')])
                   value="{{ old('tinggi_genangan') }}" required>
            @error('tinggi_genangan')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>
    </div>

    <div class="mb-3">
        <label for="tanggal_kejadian" class="form-label">Tanggal Kejadian</label>
        <input id="tanggal_kejadian" type="date" name="tanggal_kejadian"
               @class(['form-control', 'is-invalid' => $errors->has('tanggal_kejadian')])
               value="{{ old('tanggal_kejadian') }}" required>
        @error('tanggal_kejadian')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <button type="submit" class="btn btn-primary">
        <i class="fas fa-paper-plane me-1"></i>Kirim Laporan
    </button>
</form>
```

Form memiliki empat isian, yaitu nama pelapor, lokasi kejadian, tinggi genangan dalam sentimeter, dan tanggal kejadian. Form ini tidak memerlukan login, sesuai kebutuhan BPBD agar warga tetap bisa melapor.

Kartu laporan dipisahkan ke partial resources/views/partials/laporan-card.blade.php supaya bisa dipakai berulang di halaman daftar.

```
Untitled-1

@php
    $status = $laporan->tinggi_genangan < 30
        ? 'Waspada'
        : ($laporan->tinggi_genangan <= 70 ? 'Siaga' : 'Bahaya');

    $badge = $status === 'Waspada'
        ? 'bg-warning text-dark'
        : ($status === 'Siaga' ? 'bg-orange text-dark' : 'bg-danger');
@endphp

<div class="col">
    <div class="card h-100">
        <div class="card-body">
            <div class="d-flex justify-content-between align-items-start mb-2">
                <h5 class="card-title mb-0">{{ $laporan->nama_pelapor }}</h5>
                <span class="badge {{ $badge }}">{{ $status }}</span>
            </div>

            <p class="mb-1">
                <i class="fas fa-map-marker-alt me-1"></i>{{ $laporan->lokasi }}
            </p>
            <p class="mb-1">
                <i class="fas fa-ruler-vertical me-1"></i>{{ $laporan->tinggi_genangan }} cm
            </p>
            <p class="mb-0 text-muted">
                <i class="fas fa-calendar-day me-1"></i>{{ $laporan->tanggal_kejadian-
>format('d-m-Y') }}
            </p>
        </div>
    </div>
</div>
```

Partial menentukan status genangan dari nilai tinggi_genangan. Nilai di bawah 30 memakai badge Waspada, nilai 30 sampai 70 memakai badge Siaga, dan nilai di atas 70 memakai badge Bahaya. Warna badge mengikuti status agar mudah dibaca.

Halaman daftar laporan mengambil data dari controller dan menampilkannya sebagai kumpulan kartu.

```
Untitled-1

@extends('layouts.app')

@section('title', 'Daftar Laporan Banjir')

@section('content')
    <div class="d-flex justify-content-between align-items-center mb-3">
        <div>
            <h3 class="mb-0">Daftar Laporan Banjir</h3>
            <p class="text-muted mb-0">Halaman ini hanya dapat diakses oleh petugas yang sudah
login.</p>
        </div>
        <a href="{{ route('laporan.create') }}" class="btn btn-primary">
            <i class="fas fa-plus me-1"></i>Tambah Laporan
        </a>
    </div>

    @if($laporans->isEmpty())
        <div class="alert alert-info">Belum ada laporan yang masuk.</div>
    @else
        <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-3">
            @foreach($laporans as $laporan)
                @include('partials.laporan-card', ['laporan' => $laporan])
            @endforeach
        </div>
    @endif
@endsection
```

Halaman ini berada di dalam grup middleware auth. Jika pengguna belum login, Laravel mengarahkan request ke halaman login sebelum view daftar laporan dirender. Halaman konfirmasi setelah penyimpanan menampilkan pesan berhasil, detail laporan, dan tautan kembali ke form atau ke daftar laporan.

3.5 Membuat Authentication Manual

Authentication dibuat manual tanpa Laravel Breeze. Dua controller dibuat di folder app/Http/Controllers/Auth, yaitu RegisterController dan LoginController. Folder tersebut dibuat lebih dulu, lalu masing-masing controller dibuat dengan perintah artisan.

RegisterController menangani form register dan penyimpanan akun baru.

Untitled-1

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;

class RegisterController extends Controller
{
    public function create()
    {
        return view('auth.register');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect()->route('laporan.index');
    }
}
```

Method create() menampilkan view auth.register. Method store() memvalidasi nama, email yang unik, dan password yang dikonfirmasi. Password di-hash dengan Hash::make() sebelum disimpan, event Registered dipicu, lalu Auth::login() membuat session untuk akun baru. Pengguna diarahkan ke halaman daftar laporan.

LoginController menangani form login, proses autentikasi, dan logout.

Untitled-1

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class LoginController extends Controller
{
    public function create()
    {
        return view('auth.login');
    }

    public function store(Request $request)
    {
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required'],
        ]);

        if (Auth::attempt($credentials, $request->remember)) {
            $request->session()->regenerate();

            return redirect()->intended('/laporan');
        }

        return back()->withErrors([
            'email' => 'Email atau password yang Anda masukkan salah.',
        ]);
    }

    public function destroy(Request $request)
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

Method store() memvalidasi email dan password, lalu memanggil Auth::attempt() dengan kredensial tersebut. Jika cocok, session diregenerasi untuk mencegah session fixation dan pengguna diarahkan ke halaman yang sebelumnya ingin dibuka atau ke /laporan. Jika gagal, pengguna kembali ke halaman login dengan pesan galat. Method destroy() menjalankan Auth::logout(), mengosongkan session dengan invalidate(), mengganti token CSRF dengan regenerateToken(), lalu mengarahkan pengguna ke halaman utama.

View register dan login memakai layout yang sama. Form register berisi nama, email, password, dan konfirmasi password. Form login berisi email, password, dan checkbox Remember Me. Keduanya memakai @csrf, old(), dan @error seperti form pelaporan.

3.6 Mengatur Route dan Middleware Auth

Route authentication ditulis langsung di routes/web.php. Route GET /register dan POST /register memakai RegisterController. Route GET /login dan POST /login memakai LoginController. Route POST /logout memakai method destroy() pada LoginController. Route GET /laporan dan GET /profil dimasukkan ke dalam grup Route::middleware('auth')->group(...), sedangkan route form dan penyimpanan laporan tetap di luar grup.

Middleware auth memeriksa session sebelum request sampai ke controller. Jika session login masih ada, request diteruskan. Jika tidak ada, pengguna diarahkan ke route bernama login. Karena itu, route GET /login wajib diberi nama login. Tanpa nama tersebut, Laravel akan memunculkan galat Route [login] not defined.

3.7 Membuat Halaman Profil Petugas

Halaman profil dibuat dengan ProfilController dan view resources/views/petugas/profil.blade.php. Controller hanya mengembalikan view, sedangkan data pengguna diambil langsung dari facade Auth di dalam view.

```
Untitled-1

@extends('layouts.app')

@section('title', 'Profil Petugas')

@section('content')
    <div class="row justify-content-center">
        <div class="col-lg-7">
            <div class="card">
                <div class="card-header bg-white d-flex justify-content-between align-items-
center">
                    <h4 class="mb-0">Profil Petugas</h4>
                    <span class="badge bg-primary">Petugas BPBD</span>
                </div>
                <div class="card-body">
                    <dl class="row mb-0">
                        <dt class="col-sm-4">Nama</dt>
                        <dd class="col-sm-8">{{ Auth::user()->name }}</dd>

                        <dt class="col-sm-4">Email</dt>
                        <dd class="col-sm-8">{{ Auth::user()->email }}</dd>

                        <dt class="col-sm-4">Akun dibuat</dt>
                        <dd class="col-sm-8">
                            {{ Auth::user()->created_at->format('d-m-Y H:i') }}
                            ({{ Auth::user()->created_at->diffForHumans() }})
                        </dd>
                    </dl>
                </div>
            </div>
        </div>
    </div>
@endsection
```

View menampilkan Auth::user()->name, Auth::user()->email, dan Auth::user()->created_at. Waktu pembuatan akun diformat dengan format('d-m-Y H:i') dan diffForHumans() agar mudah dibaca. Halaman ini berada di dalam grup middleware auth, sehingga hanya bisa dibuka oleh petugas yang sudah login.

3.8 Membuat PetugasSeeder

Akun petugas dummy dibuat dengan seeder agar pengujian login tidak perlu mendaftar satu per satu. File database/seeders/PetugasSeeder.php dibuat dengan php artisan make:seeder PetugasSeeder, lalu diisi dengan kode berikut.

Untitled-1

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Faker\Factory as Faker;

class PetugasSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $faker = Faker::create('id_ID');

        for ($i = 0; $i < 5; $i++) {
            DB::table('users')->insert([
                'name' => $faker->name,
                'email' => $faker->unique()->safeEmail,
                'password' => Hash::make('password'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }

        // Akun admin BPBD manual
        DB::table('users')->insert([
            'name' => 'Admin BPBD',
            'email' => 'admin@bpbd.go.id',
            'password' => Hash::make('admin123'),
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    }
}
```

Faker::create('id_ID') menghasilkan nama dan email dengan format Indonesia. Perulangan for mengisi lima akun dummy dengan password 'password' yang sudah di-hash. Seeder juga menambahkan satu akun admin BPBD dengan email admin@bpbd.go.id dan password admin123. Password akun admin tetap di-hash sebelum disimpan.

Selain PetugasSeeder, saya membuat LaporanSeeder untuk mengisi 20 laporan contoh. Kedua seeder didaftarkan pada DatabaseSeeder agar bisa dijalankan bersama.

Untitled-1

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        $this->call([
            LaporanSeeder::class,
            PetugasSeeder::class,
        ]);
    }
}
```

Seeder dijalankan dengan php artisan db:seed. Hasilnya tabel users berisi lima akun dummy dan satu akun admin. Saya juga mendaftarkan satu akun melalui halaman register untuk pengujian, sehingga tabel users berisi tujuh baris. Tabel laporan berisi 20 laporan contoh, ditambah satu laporan yang saya kirim melalui form untuk pengujian, sehingga totalnya 21 baris.

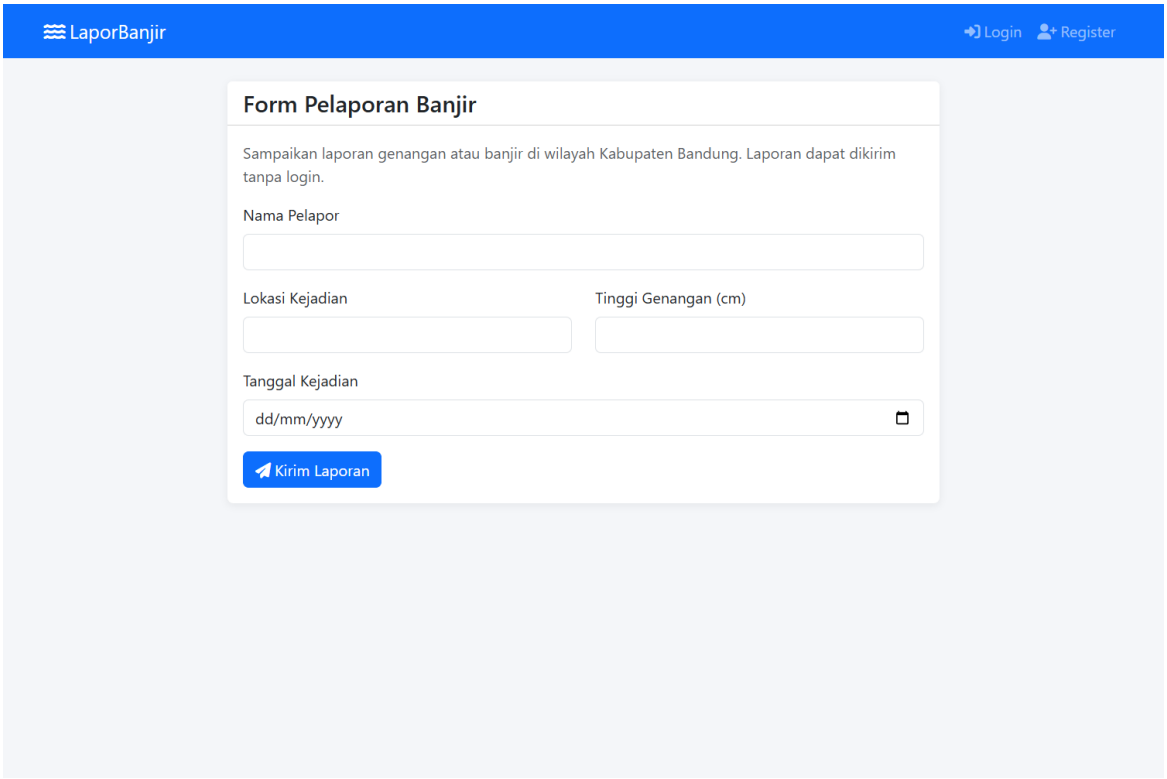
3.9 Menjalankan dan Menguji Aplikasi

Server dijalankan dengan php artisan serve di alamat http://127.0.0.1:8000. Pengujian pertama membuka form pelaporan di / tanpa login, mengisi nama Andhika Yuridho, lokasi Dayeuhkolot, tinggi genangan 45 cm, dan tanggal 20 Oktober 2026, lalu menekan tombol Kirim Laporan. Halaman konfirmasi menampilkan pesan berhasil dan status Siaga.

Pengujian kedua membuka /laporan sebelum login. Middleware auth mengarahkan browser ke /login. Setelah register akun baru, pengguna langsung diarahkan ke /laporan dan navbar berubah menjadi Daftar Laporan, Profil Petugas, dan Logout. Tombol Logout mengembalikan pengguna ke halaman utama. Login dengan password salah menampilkan pesan galat, sedangkan login dengan akun admin BPBD atau akun dummy berhasil masuk ke daftar laporan. Halaman /profil menampilkan nama, email, dan waktu pembuatan akun. Hasil pengujian ditunjukkan pada Gambar 1 sampai Gambar 11.

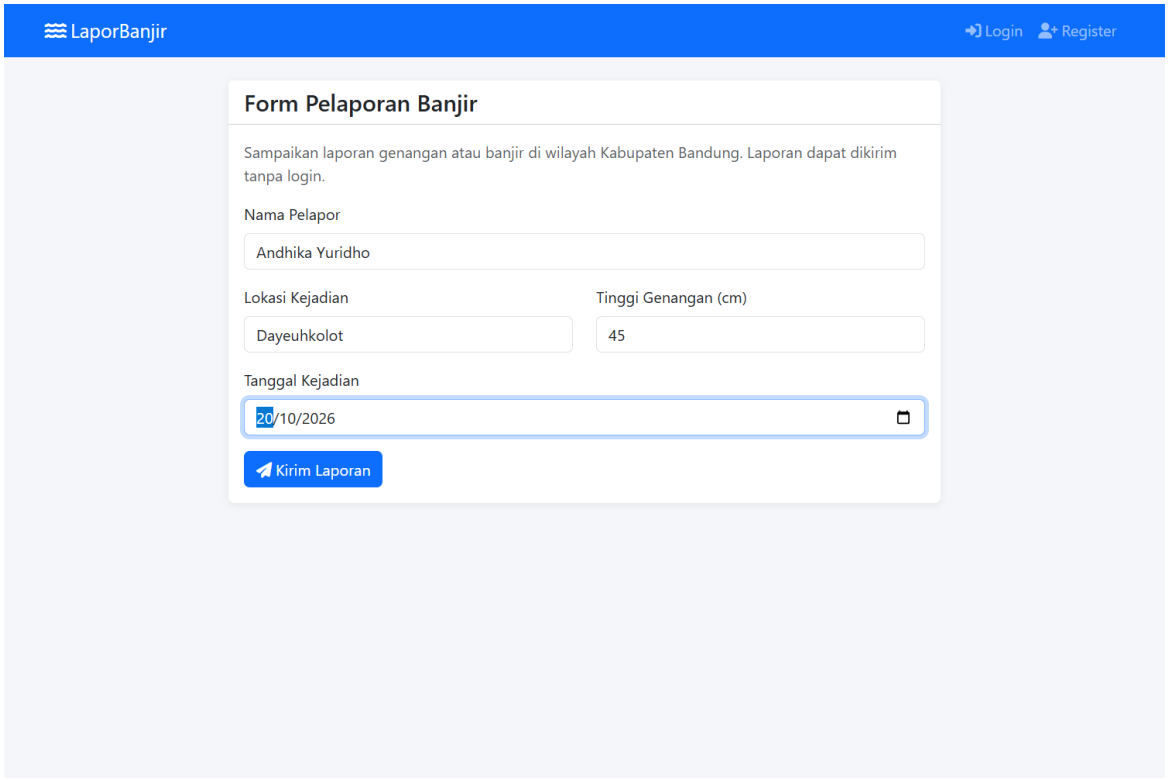
4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi LaporBanjir yang berjalan melalui php artisan serve pada alamat http://127.0.0.1:8000. Karena project ini memakai SQLite, data tabel users dan laporan ditampilkan melalui query sqlite3, bukan phpMyAdmin.



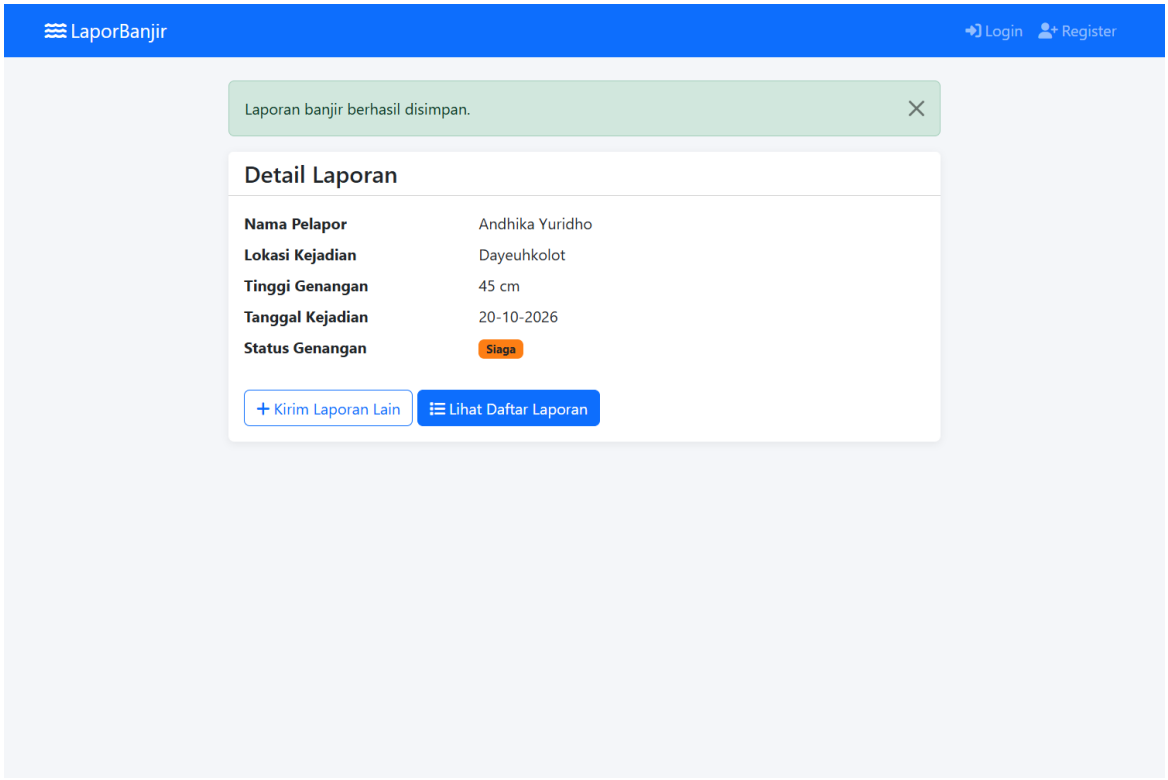
Gambar 1. Halaman form pelaporan pada alamat /

Gambar 1 menampilkan form pelaporan yang dapat dibuka tanpa login. Navbar hanya menampilkan menu Login dan Register karena pengguna belum memiliki session.



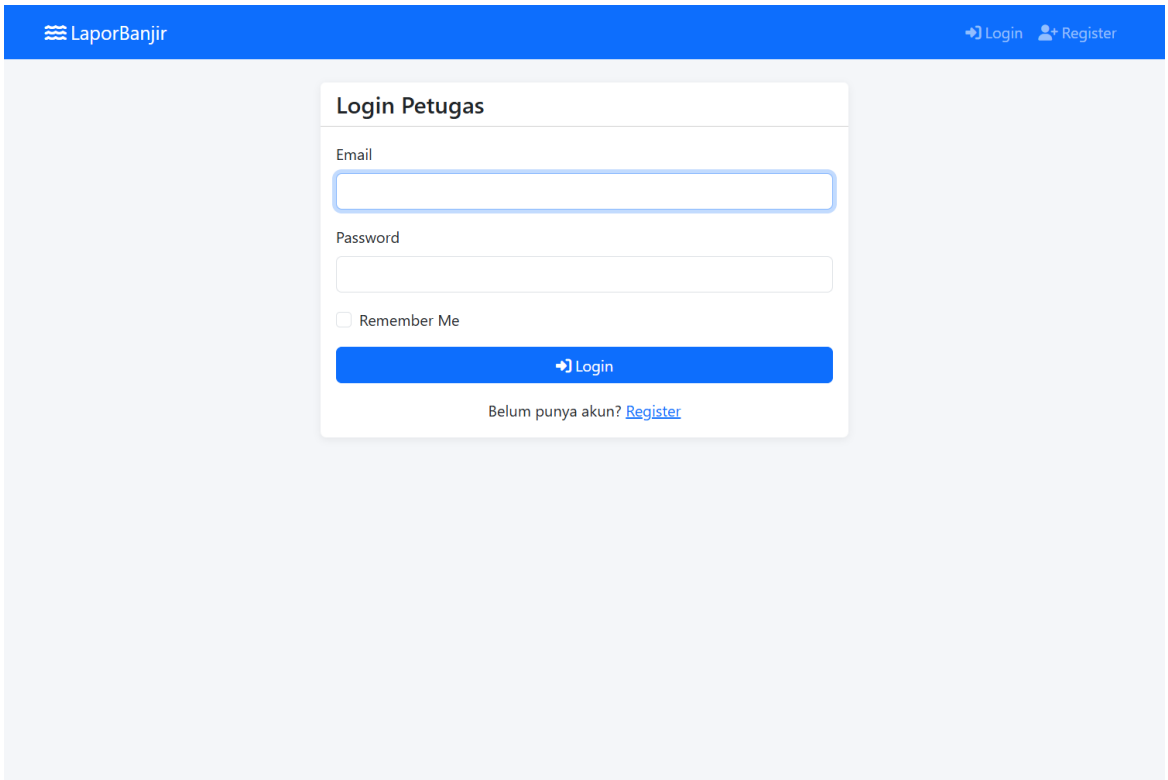
Gambar 2. Form pelaporan yang sudah diisi data warga

Gambar 2 memperlihatkan form yang sudah diisi nama Andhika Yuridho, lokasi Dayeuhkolot, tinggi genangan 45 cm, dan tanggal kejadian 20 Oktober 2026. Form ini tetap berada di halaman publik.



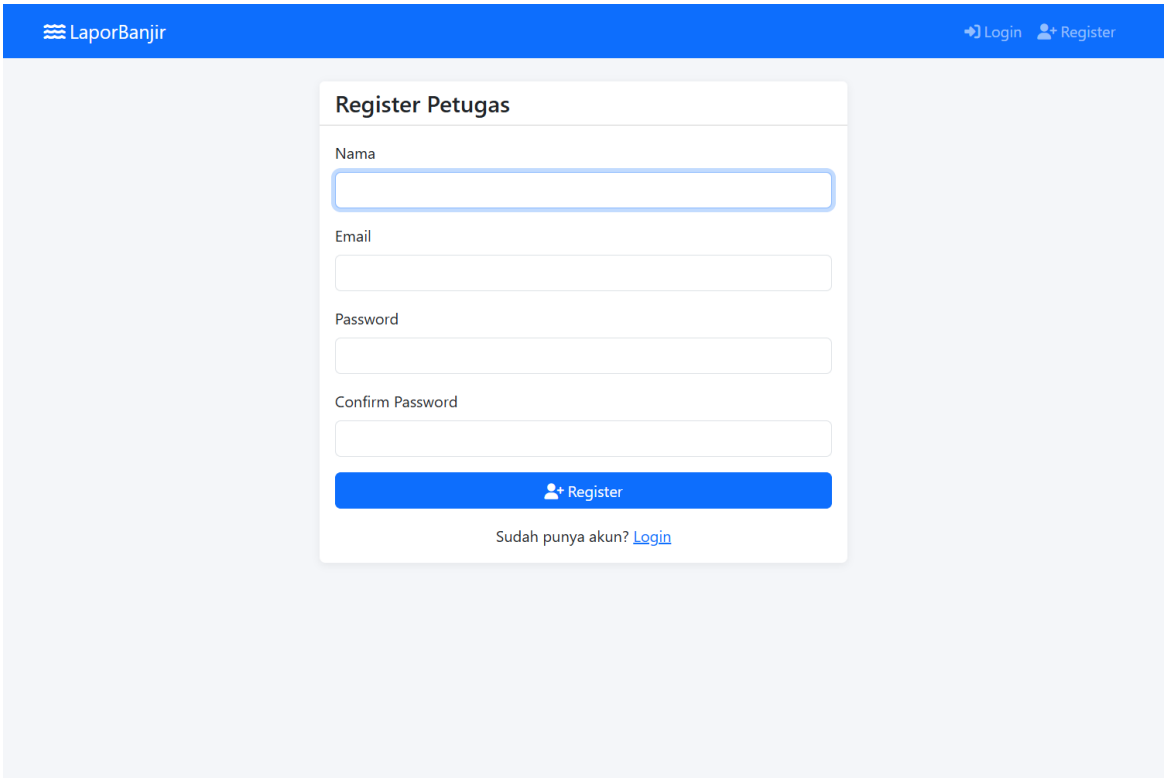
Gambar 3. Halaman konfirmasi setelah laporan disimpan

Gambar 3 menampilkan pesan Laporan banjir berhasil disimpan beserta detail laporan dan status Siaga. Data yang tampil berasal dari baris yang baru disimpan ke tabel laporan.



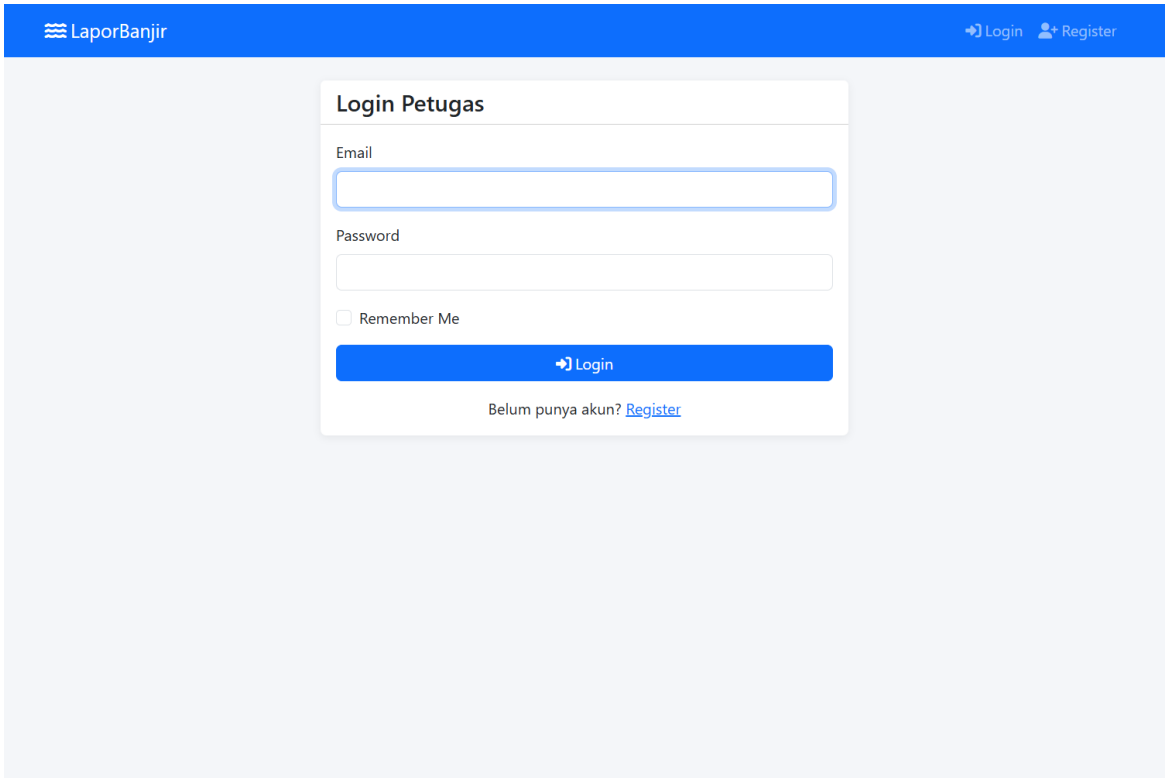
Gambar 4. Halaman login yang muncul setelah /laporan dibuka sebelum login

Gambar 4 diambil setelah alamat /laporan dibuka tanpa login. Middleware auth mengarahkan browser ke /login, sehingga halaman daftar laporan tidak sempat dirender.



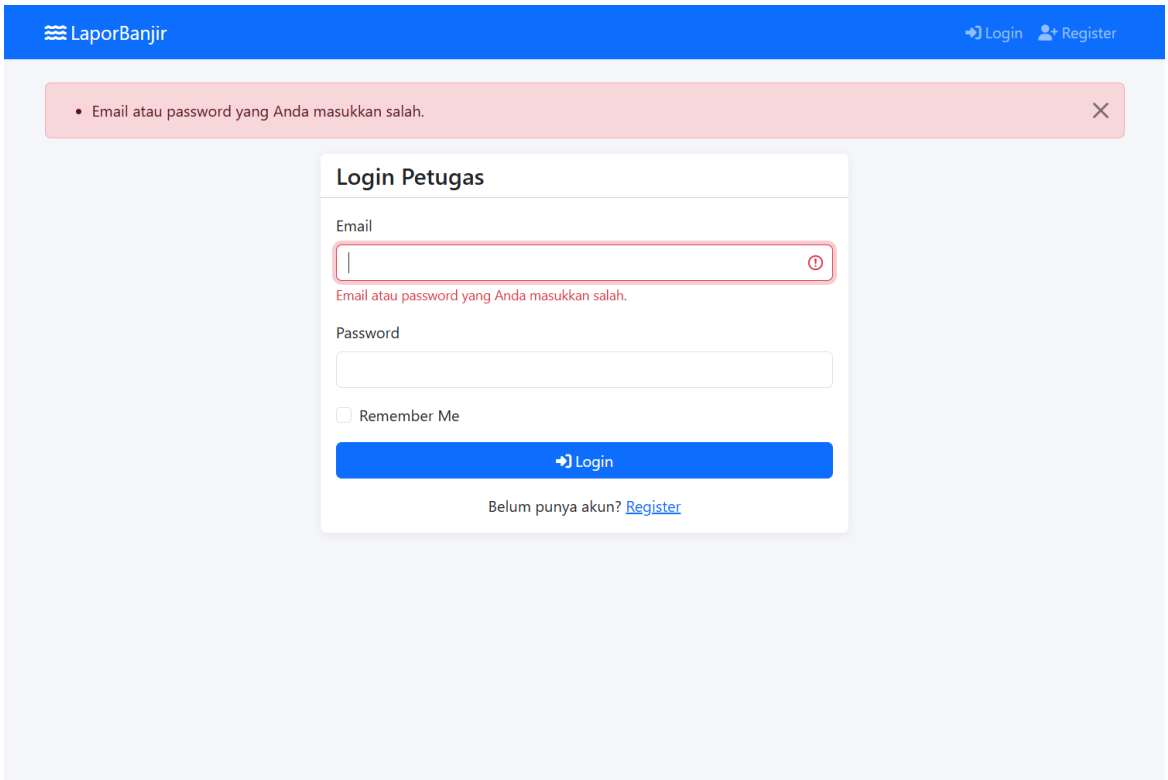
Gambar 5. Halaman register petugas pada alamat /register

Gambar 5 menampilkan form register dengan kolom nama, email, password, dan konfirmasi password. Akun yang didaftarkan di sini langsung dipakai untuk masuk ke halaman daftar laporan.



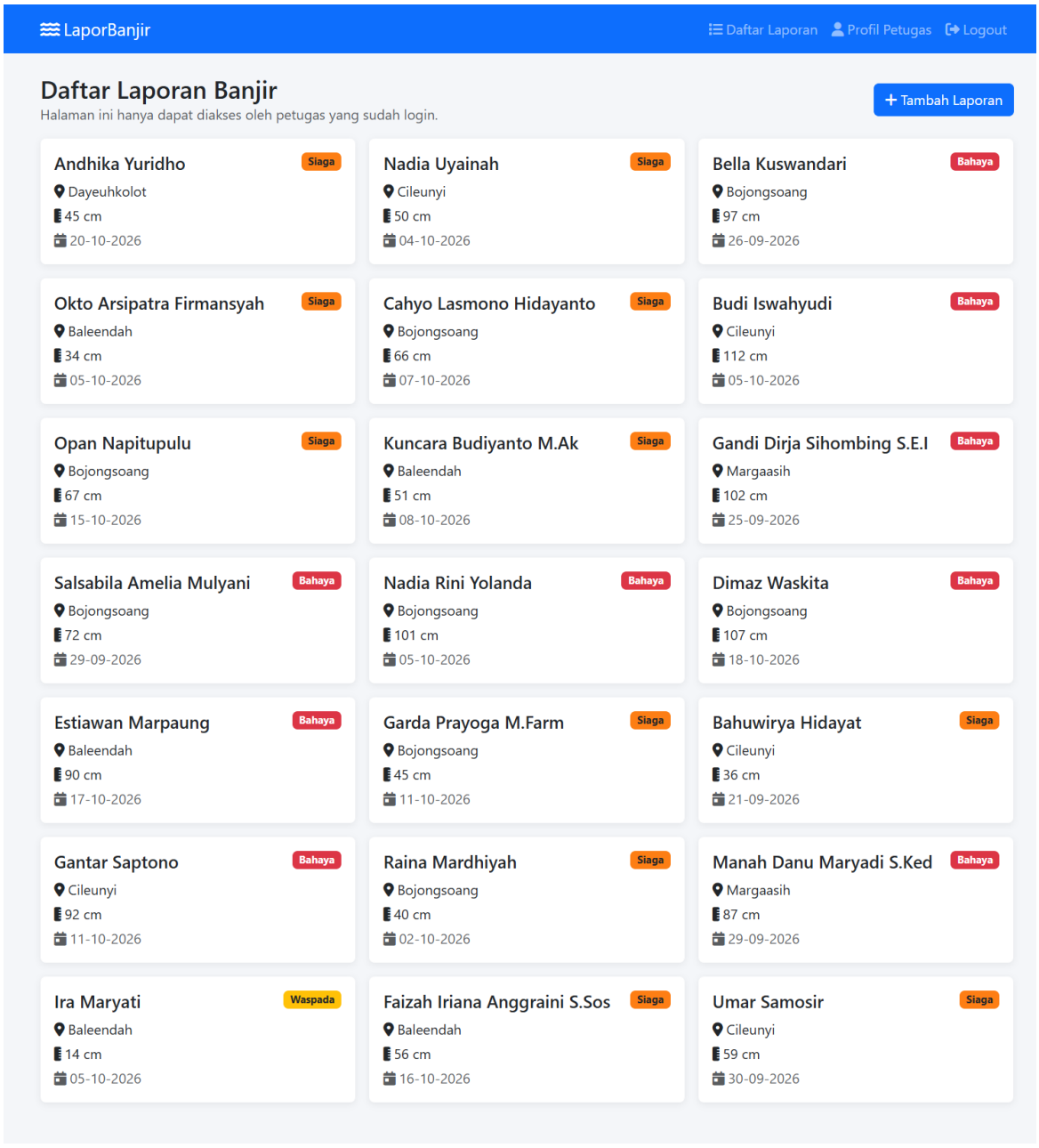
Gambar 6. Halaman login petugas pada alamat /login

Gambar 6 menampilkan form login dengan kolom email, password, dan checkbox Remember Me.



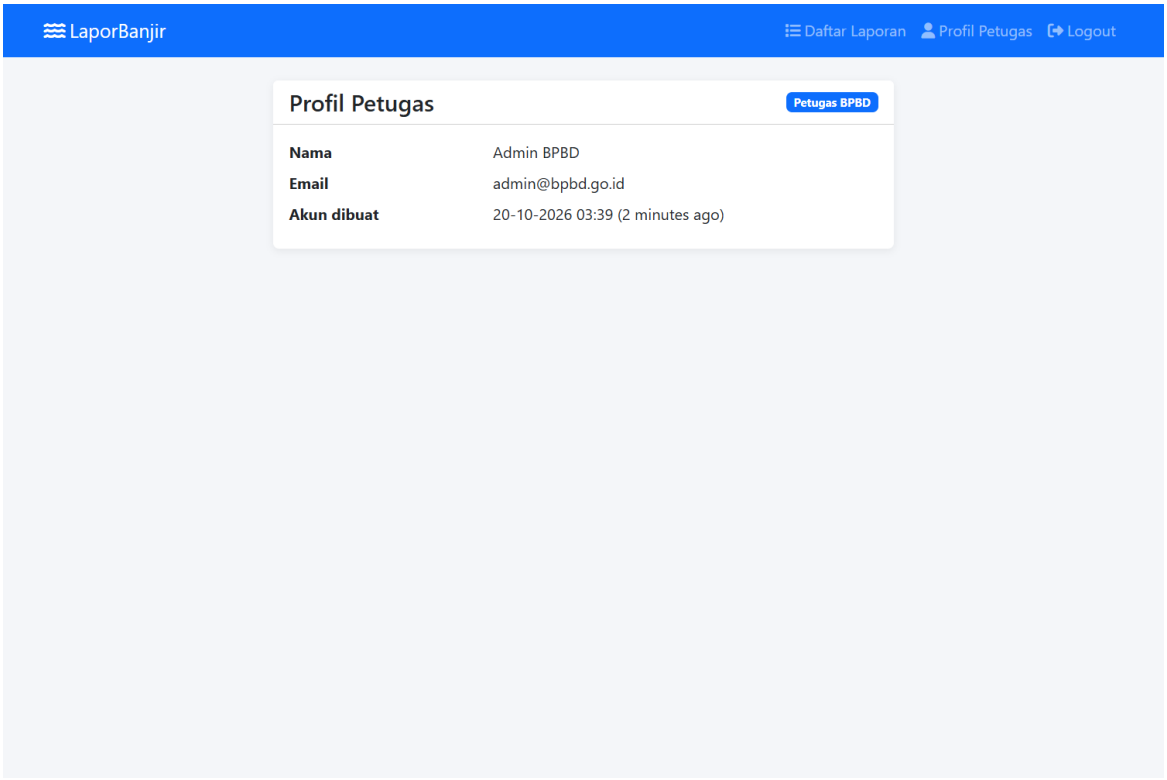
Gambar 7. Pesan galat ketika login memakai password yang salah

Gambar 7 memperlihatkan pesan Email atau password yang Anda masukkan salah. Pesan ini berasal dari method store() pada LoginController setelah Auth::attempt() gagal mencocokkan kredensial.



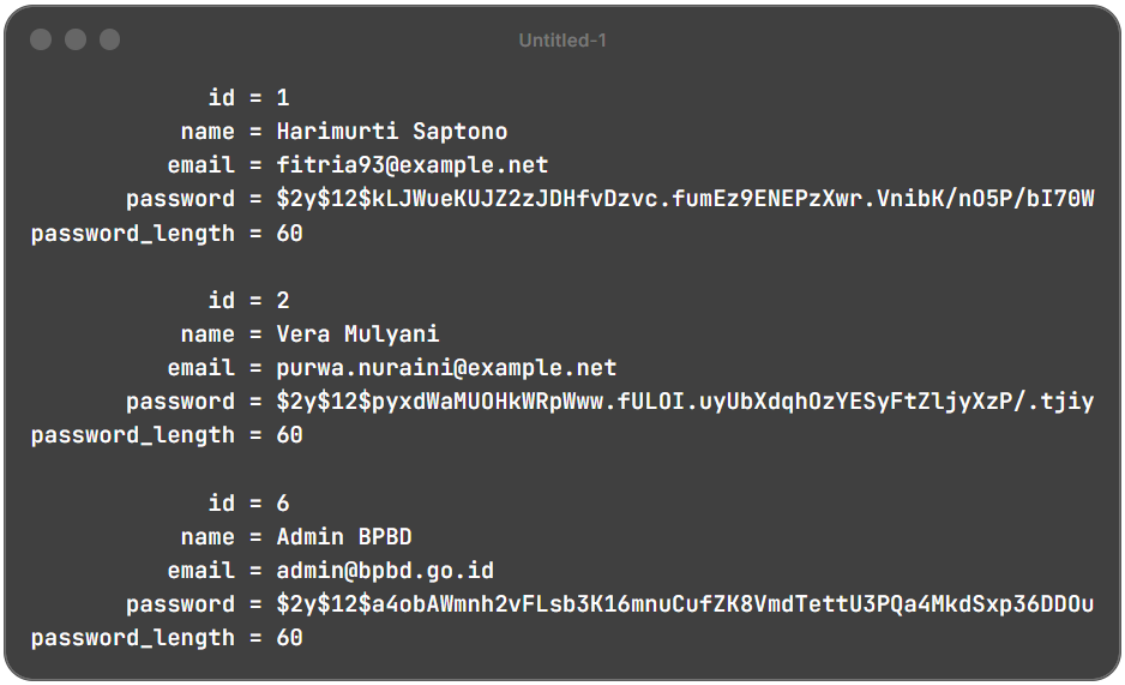
Gambar 8. Halaman daftar laporan setelah login

Gambar 8 menampilkan kartu laporan yang hanya bisa dibuka oleh petugas yang sudah login. Setiap kartu memuat nama pelapor, lokasi, tinggi genangan, tanggal kejadian, dan badge status.



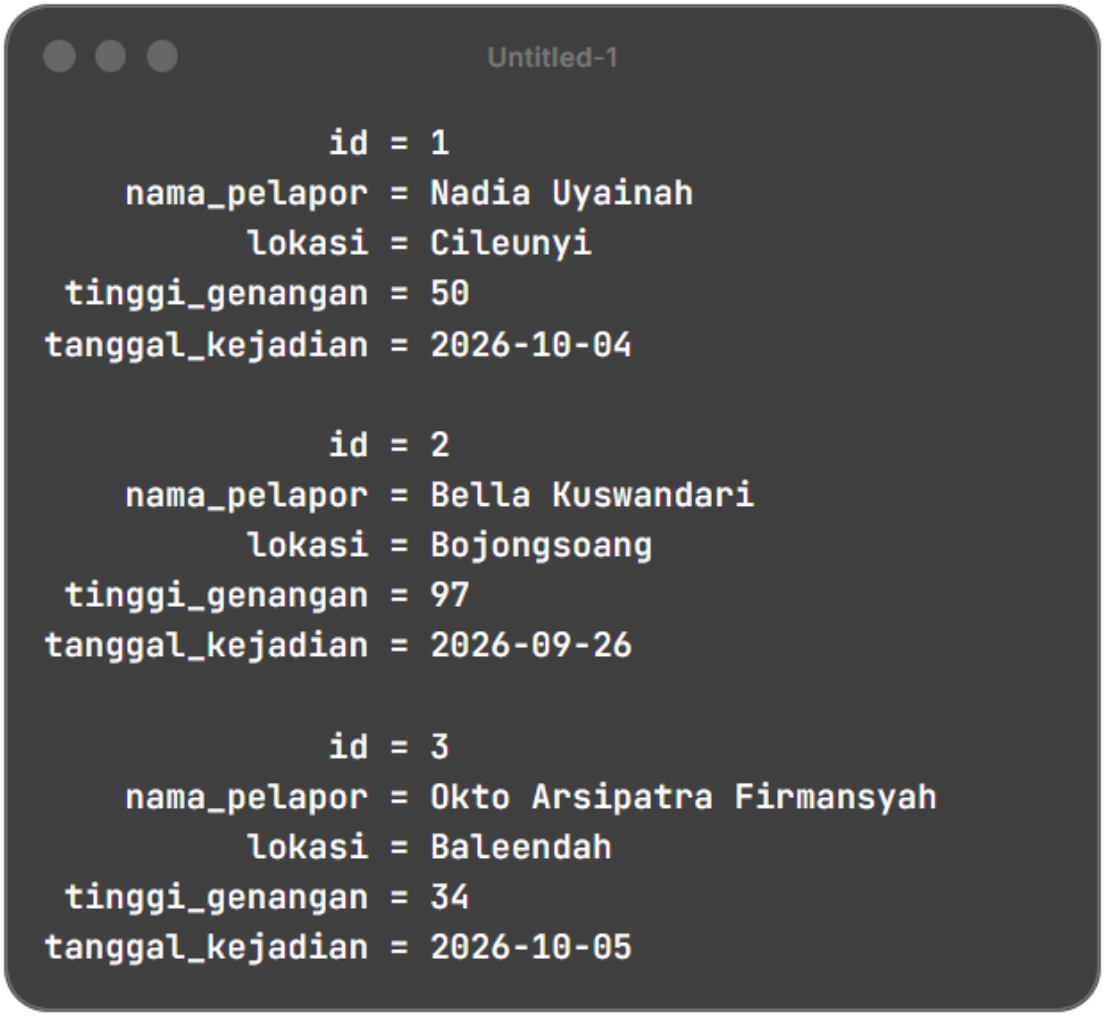
Gambar 9. Halaman profil petugas yang sedang login

Gambar 9 menampilkan nama, email, dan waktu pembuatan akun petugas. Data diambil dari Auth::user() sehingga selalu sesuai dengan akun yang sedang login.



Gambar 10. Data tabel users pada database SQLite

Gambar 10 menampilkan sebagian isi tabel users. Kolom password berisi hash bcrypt yang diawali $2y$12$ dengan panjang 60 karakter, bukan password asli. Akun admin@bpbd.go.id dan akun dummy hasil seeder dapat dipakai untuk login.



Gambar 11. Data tabel laporan pada database SQLite

Gambar 11 menampilkan sebagian isi tabel laporan hasil seeder. Kolom tinggi_genangan dan tanggal_kejadian terisi sesuai data yang dibuat Faker, dan seluruh baris bisa ditampilkan kembali di halaman daftar laporan.

5. Analisis dan Pembahasan

Perbedaan paling terasa antara LaporBanjir sebelum dan sesudah memakai authentication ada pada halaman daftar laporan. Sebelumnya, siapa pun yang mengetahui alamat /laporan bisa melihat data warga yang melapor. Setelah route tersebut dilindungi middleware auth, pengguna yang belum login diarahkan ke /login. Form pelaporan di / dan proses POST /laporan tetap

berada di luar middleware karena warga tidak boleh dipaksa membuat akun hanya untuk melapor.

Authentication manual dipilih karena tugas praktikum memang meminta fitur dibuat tanpa Breeze. Seluruh alur terlihat langsung di kode: RegisterController memvalidasi input dan memanggil Hash::make(), LoginController memanggil Auth::attempt(), dan logout menjalankan Auth::logout(). Dengan cara ini saya bisa melihat peran masing-masing komponen, sedangkan Breeze akan membuat route, controller, dan view secara otomatis sehingga prosesnya kurang terlihat.

Session menjadi penghubung antara proses login dan request berikutnya. Laravel menyimpan ID session pada cookie di browser dan data session di server. Pada project ini SESSION_DRIVER diisi database, sehingga data session tersimpan di tabel sessions. Setelah login berhasil, session()->regenerate() membuat ID baru agar ID lama tidak bisa dipakai ulang. Middleware auth membaca session tersebut untuk menentukan apakah pengguna sudah login.

Password disimpan dalam bentuk hash bcrypt, bukan teks asli. Hash::make() menghasilkan nilai yang diawali $2y$12$ dengan panjang 60 karakter, dan nilai itu tidak bisa dikembalikan menjadi password asli. Saat login, Auth::attempt() mencocokkan password yang diketik dengan hash di database. Seeder juga memakai Hash::make() sebelum menyimpan password akun dummy dan akun admin.

Middleware auth dan nama route login saling bergantung. Middleware mengarahkan pengguna yang belum login ke route bernama login. Tanpa nama tersebut, Laravel akan memunculkan galat Route [login] not defined. Pada routes/web.php, route GET /login memang diberi nama login, sehingga pengalihan berjalan.

Directive @auth dan @endauth dipakai untuk menampilkan menu yang berbeda. Pengguna yang belum login melihat menu Login dan Register, sedangkan pengguna yang sudah login melihat menu Daftar Laporan, Profil Petugas, dan Logout. Tombol Logout dibuat dalam form POST dengan @csrf karena route logout memakai method POST, sehingga proses keluar tidak bisa dipicu hanya dengan membuka tautan.

PetugasSeeder memakai Faker dengan locale id_ID untuk menghasilkan lima akun dummy dengan nama dan email Indonesia. Password seluruh akun dummy adalah 'password' dan disimpan dalam bentuk hash. Akun admin BPBD dibuat dengan email admin@bpbd.go.id dan password admin123 agar mudah dipakai saat demonstrasi. Kedua jenis akun berhasil dipakai untuk login ke halaman daftar laporan.

Ada satu penyesuaian yang saya lakukan terhadap modul. Modul memakai MySQL dan phpMyAdmin, sedangkan project ini memakai SQLite. Karena itu, pengecekan tabel users dan laporan dilakukan melalui query sqlite3, bukan phpMyAdmin. Perbedaan ini tidak mengubah alur authentication, karena Laravel memakai Eloquent dan Query Builder yang sama untuk kedua database.

Folder praktikum6-ta yang saya pakai juga berisi project Laravel baru, bukan salinan project LaporBanjir dari Modul 4. Karena itu, fitur dasar LaporBanjir saya siapkan kembali pada subbab 3.1 sampai 3.4 sebelum authentication ditambahkan. Struktur data, route, dan tampilan dibuat mengikuti laporan TA Modul 4 supaya alurnya tetap sama. Tidak ada galat berarti selama pengujian. Satu perilaku yang perlu diperhatikan adalah halaman konfirmasi dirender langsung oleh method store(), sehingga browser masih berada di alamat /laporan setelah POST. Jika halaman di-refresh, browser akan menawarkan pengiriman ulang form.

6. Kesimpulan

Berdasarkan praktikum tugas Modul 6 ini, aplikasi LaporBanjir berhasil dilengkapi dengan authentication manual tanpa Laravel Breeze. Fitur register, login, dan logout berjalan dengan validasi input, Hash::make(), Auth::attempt(), dan Auth::logout(). Halaman daftar laporan dan profil petugas hanya dapat dibuka setelah login, sedangkan form pelaporan tetap dapat diakses warga tanpa akun.

Navigasi berubah sesuai status login melalui directive @auth dan @endauth. Halaman profil menampilkan nama, email, dan waktu pembuatan akun dari Auth::user(). PetugasSeeder dengan Faker berhasil membuat lima akun dummy dan satu akun admin BPBD dengan password yang sudah di-hash. Pengujian di browser menunjukkan redirect ke login, register yang langsung membuat session, pesan galat saat password salah, logout yang mengakhiri session, serta login yang berhasil memakai akun hasil seeder.

7. Link Repository Github

https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/pabw-6/praktikum6-ta